



Yesterday 8:25 AM

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\hp\OneDrive\Desktop\se_level1> git clone https://github.com/SETAPESU26/12_maze_runner.git
Cloning into '12_maze_runner'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 11 (delta 0), reused 11 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), done.
PS C:\Users\hp\OneDrive\Desktop\se_level1> cd '12_maze_runner'
>> ^C
PS C:\Users\hp\OneDrive\Desktop\se_level1> cd 12_maze_runner
PS C:\Users\hp\OneDrive\Desktop\se_level1\12_maze_runner>
```



0d9226fb-455e-4816-80...
PDF

[SETAPESU26/12_maze_runner.git](https://github.com/SETAPESU26/12_maze_runner.git) hey this lab 4 i have send instruction what we do i seng githubb link of my problem and i have clone part so give next step by step give what do fron search

Yes. You have completed the **first part correctly**:

PowerShell

```
git clone https://github.com/SETAPESU26/12_maze_runner.git
cd 12_maze_runner
```

Your Lab 4 instruction says you now need to understand the README, run the Python game, record a **10-second video before changes**, complete all 4 tasks using Vibe Coding prompts, record another video, and push your completed work to your own repository.

0d9226fb-455e-4816-8055-265a7af...

I checked your assigned repository. It is a **Pygame Maze Runner** and the README has exactly **4 tasks**. GitHub

Do this now — Step by Step

STEP 1 — Check the files

You are currently here:

```
PS C:\Users\hp\OneDrive\Desktop\se_level1\12_maze_runner>
```

Run:

PowerShell

```
dir
```

You should see something similar to:

```
game  
.gitignore  
README.md  
main.py  
requirements.txt
```

The repository structure confirms these files/folders.  GitHub

STEP 2 — Check Python

Run:

PowerShell

```
python --version
```

You should get something like:

```
Python 3.x.x
```

Then:

PowerShell

```
pip --version
```

If both work → continue.

STEP 3 — Install required packages

Run:

PowerShell

```
pip install -r requirements.txt
```

The repository README specifically gives this setup command.  GitHub

Wait until installation finishes.

If you get:

```
Successfully installed pygame...
```

then it is successful.

STEP 4 — Run the ORIGINAL game

Before changing anything, run:

PowerShell

```
python main.py
```

The README says `python main.py` starts the game.  GitHub

A Maze Runner window should open.

Test the original controls

Key	Function
W / ↑	Move up
S / ↓	Move down
A / ←	Move left
D / →	Move right
R	Generate new maze

These are the original controls listed in the README.  GitHub

STEP 5 — RECORD BEFORE VIDEO

Do this before making ANY code changes.

Record approximately 10 seconds.

For example:

1. Start the game.
2. Move the player using `W/A/S/D`.
3. Show the maze.
4. Press `R` to demonstrate a new maze.
5. Stop recording.

Save it as:

```
before.mp4
```

This is required by your Lab 4 instructions.

0d9226fb-455e-4816-8055-265a7af...

Important: Don't modify the code before recording this video.

STEP 6 — Understand the 4 tasks

Your README requires these four features:

Task 1 — Shortest Path Hint

Press:

```
H
```

The shortest path from the player/start to the exit should be highlighted.

The README specifically suggests using **BFS**.  [GitHub](#)

Task 2 — Fog of War

Only cells within **3 cells around the player** should be visible.

Everything else should be dark.  [GitHub](#)

Task 3 — Timer Leaderboard

After completing the maze:

- Record completion time.
- Keep top 5 times.
- Sort them.
- Display leaderboard.
- Save it permanently in:

```
leaderboard.json
```



Task 4 — Difficulty

At the beginning, show:

```
Easy    → 10 x 8
Medium  → 15 x 13
Hard    → 20 x 18
```

User selects one before the maze starts. 

STEP 7 — Now use Vibe Coding

This is the important part of your Lab.

Your teacher wants you to **write prompts and use an AI coding tool to modify the code**, rather than manually implementing everything. The instructions specifically say to use prompts to fix the broken code and add the README features one by one.

```
0d9226fb-455e-4816-8055-265a7af...
```

You can use ChatGPT/Cursor/Claude/Gemini.

I recommend doing **one task at a time**.

STEP 8 — Prompt for Task 1

Open the project in your Vibe Coding tool.

If using VS Code:

```
PowerShell
```

```
code .
```

Then give the AI this prompt:

```
Go through the entire Maze Runner project and understand the existing code before making
```

```
Implement Task 1 from README.md: Shortest Path Hint.
```

```
Requirements:
```

- Add a BFS-based shortest path solver.
- When the user presses H, calculate the shortest path from the player's current position

- Highlight the shortest path clearly on the maze.
- Pressing H again should toggle the path off.
- Do not remove or break the existing W/A/S/D movement controls.
- Do not break the R key new-maze functionality.
- Keep the existing project structure and coding style.
- Modify only the files necessary.
- After implementing, explain which files were changed and how to test Task 1.

Let the AI modify the project.

STEP 9 — Test Task 1

Run:

PowerShell

```
python main.py
```

Test:

H

You should see the shortest path highlighted.

Then:

H

again.

The path should disappear.

If it works → continue to Task 2.

Don't immediately ask AI to do all four tasks together. Your assignment specifically asks for features to be added one by one.

0d9226fb-455e-4816-8055-265a7af...

STEP 10 — Prompt for Task 2

Give the AI:

```
Now implement Task 2 from README.md: Fog of War.
```

Requirements:

- Only reveal maze cells within a radius of 3 cells around the player.
- The rest of the maze should appear dark/hidden.
- The visible area must move with the player.
- Make sure the player and nearby maze cells remain clearly visible.
- Do not break Task 1 shortest path hint.
- Do not break W/A/S/D movement.
- Do not break the R key new maze functionality.
- Keep the existing project structure.
- Use Pygame surfaces/transparency appropriately.
- After implementation, explain how to test the fog of war.

Then test:

PowerShell

```
python main.py
```

Move around with:

W A S D

Check that the visible area follows the player.

STEP 11 — Prompt for Task 3

After Task 2 works, give:

Now implement Task 3 from README.md: Timer Leaderboard.

Requirements:

- Start a timer when a maze game begins.
- Stop the timer when the player reaches the exit.
- Record the completion time.
- Maintain only the top 5 best completion times.
- Sort the times from fastest to slowest.
- Display the leaderboard on the win screen.
- Persist the leaderboard in a JSON file named leaderboard.json.
- If leaderboard.json does not exist, create it automatically.
- Make sure invalid or corrupted leaderboard data does not crash the game.
- Do not break Tasks 1 and 2.
- Do not break W/A/S/D and R controls.
- Keep the implementation simple and consistent with the existing project.
- After implementation, explain how to test the leaderboard.

Then:

PowerShell

```
python main.py
```

Complete a maze.

Check that the completion time appears.

Then check your project folder:

PowerShell

```
dir
```

You should eventually see:

```
leaderboard.json
```

STEP 12 — Prompt for Task 4

Then:

Now implement Task 4 from README.md: Larger Maze Difficulty Tiers.

Requirements:

- Add a difficulty selection screen before the maze starts.
- Provide three selectable options:
 - Easy = 10 columns x 8 rows
 - Medium = 15 columns x 13 rows
 - Hard = 20 columns x 18 rows
- Allow the user to select the difficulty using the mouse/buttons.
- Generate the maze using the selected dimensions.
- Keep the existing movement controls.
- Keep the R key functionality to generate a fresh maze using the selected difficulty.
- Do not break Tasks 1, 2, or 3.
- Make the UI simple and consistent with the existing game.
- After implementation, explain how to test all three difficulty levels.



Test:

PowerShell

```
python main.py
```

You should get something similar to:

Choose Difficulty

[EASY]

[MEDIUM]

[HARD]

STEP 13 — Test EVERYTHING

Now test all four tasks together.

Test 1

H

Shortest path appears/disappears.

Test 2

Move:

W A S D

Fog follows player.

Test 3

Complete maze.

Check:

Completion Time

Leaderboard

Check:

leaderboard.json

Test 4

Restart game and test:

Easy

Medium

Hard

Make sure maze sizes change.

Also test R

Press:

R

A fresh maze should be generated. The README explicitly requires fresh maze generation with R .  GitHub

STEP 14 — Check Git changes

Close the game and run:

PowerShell

```
git status
```

You should see modified/new files.

Then:

PowerShell

```
git diff
```

This lets you see what the AI changed.

STEP 15 — IMPORTANT: Don't push to SETAPESU26

Your current repository is:

```
https://github.com/SETAPESU26/12_maze_runner
```

Do not push your changes back to this repository.

Your Lab instruction explicitly says:

Do not raise PR to the main repo (SETAPESU26).

0d9226fb-455e-4816-8055-265a7af...

You need your own GitHub repository/personal Lab repository.

First check:

PowerShell

```
git remote -v
```

You will probably see:

```
origin https://github.com/SETAPESU26/12_maze_runner.git
```

STEP 16 — Create your own GitHub repository

On GitHub, create a new repository under **your personal GitHub account**.

For example:

```
12_maze_runner
```

Do **not** initialize it with another README if you already have the cloned project.

Then copy your personal repository URL.

It will look like:

```
https://github.com/YOUR_USERNAME/12_maze_runner.git
```

STEP 17 — Change remote

Inside:

```
12_maze_runner
```

run:

PowerShell

```
git remote set-url origin https://github.com/YOUR_USERNAME/12_maze_runner.git
```

Replace:

```
YOUR_USERNAME
```

with your actual GitHub username.

Then check:

PowerShell

```
git remote -v
```

Make sure it shows **your account**, not:

```
SETAPESU26
```

STEP 18 — Commit your completed code

Run:

PowerShell

```
git add .
```

Then:

PowerShell

```
git commit -m "Complete Maze Runner Lab 4 tasks"
```

Then:

PowerShell

```
git push -u origin main
```

If GitHub asks you to authenticate, complete the GitHub login.

STEP 19 — Record AFTER video

Now record another approximately 10-second video.

Show:

1. Difficulty selection.
2. Maze running.
3. Fog of war.

4. Press `H` → shortest path.
5. Complete maze / leaderboard if possible.

Save:

after.mp4

The Lab requires a second video after all changes.

0d9226fb-455e-4816-8055-265a7af...

STEP 20 — Prepare your Lab-4 folder

Your Lab instructions say the Lab repo should contain a **Lab-4** folder with:

```
Lab-4/
|
├── Videos/
|   ├── before.mp4
|   └── after.mp4
|
├── Updated_Code/
|   ├── main.py
|   ├── requirements.txt
|   ├── leaderboard.json
|   └── game/
|       ├── __init__.py
|       ├── game_engine.py
|       ├── maze.py
|       └── player.py
|
└── Chat_History/
    └── chat_history.pdf
```

The required deliverables are **before/after videos**, **updated code**, and **exported chat history**.

0d9226fb-455e-4816-8055-265a7af...

Your current position

You are currently at:

- ☒ Clone repository
- ☒ Enter repository
- ☐ Install requirements
- ☐ Run original game
- ☐ Record BEFORE 10-sec video
- ☐ Task 1 – BFS shortest path